

Master Tester Agent

(MTA) — Technical Architecture & System Design

Protocol-First Adapter/Evaluator Platform · v3.0 (Draft)

Version	3.0 (Draft)
Date	August 6, 2026
Author	[Your name]
Status	Draft — for review
Reference	BRD_Master_Tester_Agent_v3.md v1.0, RESEARCH_Industry_Landscape.md
Supersedes	TSD_Master_Agent_Tester.md v0.1, TSD_Master_Tester_Agent_v2.md — retained for history, not for use

0. Document Scope & Phasing

This TSD specifies the full target architecture for MTA, consistent with the long-term vision. Not all of it is being built in the delivery scoped by BRD v3. Every subsection below is tagged with its delivery phase so a reader can tell, at a glance, what is being built now versus later:

- [Phase 1–3] — in scope for the delivery BRD v3 describes (core platform + Web/API/Agent adapters + rule-based/LLM-judge evaluation + CI/CD gate + judge calibration)
- [Future Scope — FS-xx] — specified here as the target design, but deferred; FS-xx cross-references BRD v3 §11 (Future Scope table), where the business case for building it lives

This keeps one architecture document as the durable technical reference, rather than maintaining a separate “someday” architecture that drifts out of sync with what's actually built.

1. Overview [Phase 1–3]

MTA is a protocol-first, adapter/evaluator-based platform: a small set of core services orchestrates test execution against any registered solution type, through a narrow, versioned contract that both first-party and future adapters/evaluators implement identically. This is the single architectural principle the whole system is built around — validated against Selenium Grid's WebDriver protocol, Terraform's provider RPC protocol, and pytest's pluggy hookspecs, all of which age well specifically because the core never depends on adapter internals.

The pipeline: a solution registry (catalog of every registered solution plus its BRD/TSD) feeds a requirements & test generator (turns requirements into traceable, ISO/IEC 25010-tagged test specs), which feeds a set of execution adapters (one per solution type/surface), which feed an evaluation engine (rule-based, LLM-as-judge, and — in later

phases — security, RAG-specific, memory-specific, accessibility, performance, business-rule evaluators — each emitting a confidence signal), which feeds a reporting and traceability dashboard. Two standing subsystems run alongside this pipeline rather than as pipeline stages: a judge-calibration subsystem (continuously validates the evaluation engine's own trustworthiness) and a cross-platform consistency engine (Future Scope — runs one canonical test set through every adapter a given solution is deployed behind, once ≥ 2 platform adapters exist).

FIGURE 1 — MTA CORE PLATFORM ARCHITECTURE

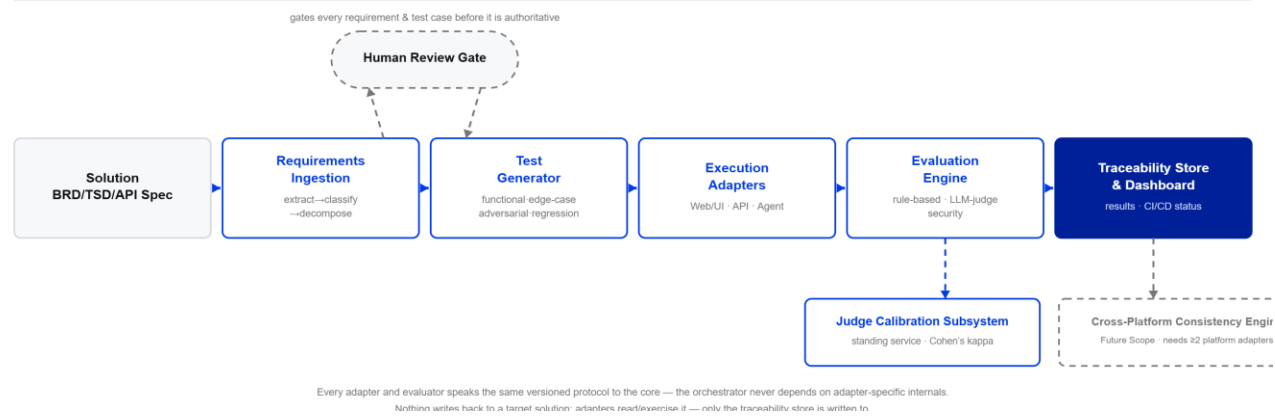


Figure 1 — MTA core platform architecture: the same pipeline described above, registry → ingestion → test generation → execution adapters → evaluation → traceability & dashboard, with the human review gate and the two standing subsystems shown alongside the stages they govern.

This section maps each business requirement to the technical requirements that satisfy it.

2. Requirement traceability (BRD → TSD)

BRD ID	Technical requirement(s)	Phase
BR-D-01–04 (Registry view)	TR-1.x (Solution Registry Service)	Phase 1
BR-D-05–08 (Traceability matrix)	TR-2.x (Requirements & Test Generator), TR-6.2	Phase 1
BR-D-09–12 (Release readiness)	TR-6.x, TR-7.x (CI/CD tiered gating)	Phase 1/3
BR-D-13–15 (Findings & review queue)	TR-2.4, TR-4.8, TR-4.10	Phase 1
BR-D-16–17 (Judge calibration view)	TR-8.x (Judge Calibration Subsystem)	Phase 3
BR-R-01–05 (Registry & ingestion)	TR-1.x, TR-2.x	Phase 1
BR-A-01–05 (Adapters)	TR-3.1 (protocol), TR-3.2/3.3/3.6 (Web/API/Agent)	Phase 2
BR-E-01–06 (Evaluation engine)	TR-4.1–4.3, TR-4.8, TR-4.10, TR-9.3	Phase 2
FS-01 (RAG)	TR-3.5, TR-4.4	Future Scope
FS-02 (Platform/bot adapters)	TR-3.4	Future Scope

BRD ID	Technical requirement(s)	Phase
FS-03 (Cross-platform consistency)	TR-5.x	Future Scope, blocked on FS-02
FS-04 (Memory validation)	TR-4.5	Future Scope
FS-05 (Performance)	TR-3.8	Future Scope
FS-06 (Accessibility)	TR-3.9	Future Scope
FS-07 (Business-rule validation)	TR-2.3 (business-rule branch), TR-4.2 (rule-engine matching)	Future Scope
FS-08 (Adaptive red-teaming)	TR-4.6 (adaptive half)	Future Scope
FS-09 (Enterprise integrations)	TR-3.7	Future Scope

3. System architecture

3.1 Solution Registry Service (TR-1) [Phase 1]

- TR-1.1 — Data store (solutions table): id, name, type [agent|rag|web|api|platform_bot|internal_tool|integration|other], owner, current_version, endpoint_ref, credentials_ref, brd_uri, tsd_uri, deployed_surfaces[], status, created_at, updated_at. deployed_surfaces[] (e.g., ["web", "teams", "slack"]) is what the Cross-Platform Consistency Engine (TR-5.x, Future Scope) will read once built; populated from Phase 1 even though nothing consumes it yet, so no schema migration is needed later.
- TR-1.2 — Registration API for onboarding a new solution of any type (see §5)
- TR-1.3 — Each solution record links to its BRD/TSD source document(s), which feed the requirements ingestion pipeline

3.2 Requirements Ingestion & Test Generator (TR-2) [Phase 1]

- TR-2.1 — Document parser ingests BRD/TSD/user stories/API specs from PDF, Word, Confluence, Notion, or a raw OpenAPI/GraphQL schema
- TR-2.2 — LLM extraction step converts prose requirements into structured requirement objects (see §11 for schema), using a multi-step decomposition (extract → classify → decompose into acceptance criteria) rather than single-shot extraction, per industry-consensus practice — single-shot extraction has no reliable published accuracy figure and must not be trusted as ground truth without the gate in TR-2.4
- TR-2.3 — Test spec generator converts each requirement object into one or more test cases (see §11 for schema): functional/UI/API requirements use scenario- or spec-driven generation (Phase 1); business-rule requirements use decision-table extraction + PICT-style combinatorial generation, with business constraints encoded as exclusion rules [Future Scope — FS-07]; AI-agent/RAG requirements use LLM-synthesized scenario generation plus production-derived golden cases (agent: Phase 2; RAG: [Future Scope — FS-01])
- TR-2.4 — Every auto-generated test case and every auto-extracted requirement object enters a human review queue; only approved items move to the active suite (satisfies BR-R-04)

- TR-2.5 — Test types generated per requirement: smoke, sanity, functional, edge-case, negative/boundary, adversarial/security (static corpus), regression, integration, API (contract + fuzz) — Phase 1–2. UI visual regression, performance, accessibility, business-rule, RAG, memory, and cross-platform types are [Future Scope], tagged per §2 above.
- TR-2.6 — Every generated test case is tagged with the ISO/IEC 25010 quality characteristic(s) it validates (Functional Suitability, Performance Efficiency, Compatibility, Interaction Capability, Reliability, Safety, Security, Maintainability, Flexibility), populating TR-4.9/TR-6.3

3.3 Execution Adapters (TR-3)

- TR-3.1 [Phase 2] — Common adapter protocol (data/RPC boundary, not shared code — per the Terraform-provider/WebDriver precedent): connect(solution_target) -> Session; send_input(session, input) -> None; get_response(session) -> RawResult; teardown(session) -> None. Adapters are versioned independently of the core orchestrator; a protocol-version handshake prevents silent breakage when either side updates. Every future adapter — Phase 2 or Future Scope — implements this same protocol; nothing below requires a core change to add.
- TR-3.2 — Web/UI adapter [Phase 2]: built on Playwright (auto-waiting, native multi-browser, CDP-based). Visual regression, cross-browser device-farm matrices, and the agentic Planner→Generator→Healer authoring pipeline are [Future Scope] refinements on top of this same adapter — Phase 2 ships functional UI automation only.
- TR-3.3 — API adapter [Phase 2]: schema/contract-driven, built around Schemathesis-style property-based fuzzing directly from an OpenAPI/GraphQL schema for positive/negative/boundary/fuzz case execution (Phase 2); Pact-style consumer-driven contract verification ([Future Scope] — not required until a second independently-deployed service consumes a Phase-1-registered API); systematic authorization-boundary testing (BOLA/BFLA sweep) ([Future Scope], sequenced early post-pilot given OWASP API's #1 risk ranking); oasdiff-style breaking-change detection between spec versions ([Future Scope])
- TR-3.4 — Platform/bot adapter [Future Scope — FS-02]: a canonical activity model (mirroring Bot Framework's Activity schema and the Botium connector pattern) with one adapter per platform (Teams, Slack). Full design retained here as the target: send_user_message(channel, payload) / get_bot_reply() per platform; Teams via TestAdapter/transcript-record-replay with explicit adaptive-card/invoke-activity fixtures; Slack via a purpose-built Bolt-receiver mock plus signed-webhook layer.
- TR-3.5 — RAG adapter [Future Scope — FS-01]: connects to a RAG pipeline's query interface and captures both the final answer and the retrieved-context set (required input to TR-4.4)
- TR-3.6 — Conversation simulator [Phase 2]: an LLM plays a scripted persona with a goal, carries on a multi-turn exchange with the target agent, and terminates on success criteria or a max-turn limit
- TR-3.7 — Enterprise integration adapter [Future Scope — FS-09]: Testcontainers-backed ephemeral brokers (Kafka/RabbitMQ/Azure Service Bus), Pact-style contract tests for message-schema agreement, WireMock-style webhook stubs, disposable SFTP/batch-file harnesses
- TR-3.8 — Performance adapter [Future Scope — FS-05]: standard HTTP/WebSocket/browser adapter on k6, plus a purpose-built streaming-LLM sub-adapter (TTFT, inter-token latency, tokens/sec, cost-per-load, spend circuit-breakers) for LLM-backed agents/RAG systems
- TR-3.9 — Accessibility adapter [Future Scope — FS-06]: built on axe-core, every finding tagged with WCAG success criterion and an automation-confidence flag so manual-review-only categories are never silently auto-passed

- TR-3.10 — Adapter failure handling [Phase 2]: adapter failures retry with backoff before being recorded as a hard failure

3.4 Evaluation Engine (TR-4)

- TR-4.1 [Phase 2] — Common evaluator protocol: `evaluate(test_case, raw_result) -> Verdict`; `Verdict = { pass_fail, confidence, coverage_note, rationale }`
- TR-4.2 [Phase 2 core; Future Scope — FS-07 for the rule-engine branch] — Rule-based evaluator: schema validation, regex/exact match, latency SLA checks ship Phase 2; decision-table/PICT expected-output matching for business-rule test cases is FS-07
- TR-4.3 [Phase 2] — LLM-as-judge evaluator: prompt template built from the requirement + expected behavior. Phase 2 ships single-judge mode; position-swapped pairwise and multi-judge “jury” (majority-vote) modes are a Phase 3+/calibration-program refinement (TR-8.4), added once per-domain calibration data justifies the added cost.
- TR-4.4 — RAG evaluator [Future Scope — FS-01]: layered retrieval/generation metrics (RAGAS-style), chunk-level attribution, claim-level hallucination/citation verification
- TR-4.5 — Memory evaluator [Future Scope — FS-04]: no mature off-the-shelf toolchain exists — built in-house, borrowing task design from LongMemEval/LoCoMo/STALE. Recall accuracy, knowledge-update correctness, staleness detection, abstention-on-false-premise, adversarial cross-session/cross-tenant leakage probes. Decomposed sub-metrics, not one blended score.
- TR-4.6 — Safety/red-team evaluator: static, OWASP-mapped adversarial corpus (prompt injection, jailbreak resistance, sensitive-information disclosure, system-prompt extraction) ships [Phase 2] per BR-E-05; adaptive, LLM-driven multi-turn attack generation (PyRIT/Giskard-style `generate→analyze→mutate`) is [Future Scope — FS-08]
- TR-4.7 — Trajectory/tool-call evaluator [Phase 2, deterministic mode only; LLM-graded semantic-match mode is Future Scope]: expected-vs-actual tool-call sequence comparison
- TR-4.8 [Phase 2] — Aggregation logic: a test is “pass” only if all applicable evaluators pass; results with confidence below a defined threshold are routed to human review rather than auto-resolved (satisfies BR-E-04)
- TR-4.9 [Phase 1] — Every evaluator verdict is tagged with the ISO/IEC 25010 quality characteristic(s) its test case validates (populated at generation time by TR-2.6)
- TR-4.10 [Phase 2] — Every evaluator emits a confidence/coverage signal alongside its verdict — one shared confidence-signal contract, one shared human-escalation mechanism (TR-4.8), so a Future Scope evaluator (accessibility, RAG, memory) plugs into the same escalation path without inventing its own

3.5 Judge Calibration Subsystem (TR-8) [Phase 3] — standing service, not a pipeline stage

- TR-8.1 — A human-labeled gold set (~100-500 examples, drawn from real production data) maintained per evaluation domain, starting with the agent-quality domain in Phase 3 and adding a domain as each corresponding evaluator ships
- TR-8.2 — Scheduled re-calibration: judge verdicts on the frozen gold set re-scored on a fixed cadence and on every underlying model/prompt version bump; agreement measured with chance-adjusted metrics (Cohen's kappa / Krippendorff's alpha), never raw percent agreement

- TR-8.3 — Drift alerting: a defined drop in agreement triggers an alert and — depending on severity — a temporary tightening of the confidence threshold (TR-4.8) until recalibrated
- TR-8.4 — Per-domain rubric tuning: each evaluation domain maintains its own calibrated rubric/judge configuration rather than sharing one “universal” judge; this is also where multi-judge “jury” mode (deferred from TR-4.3) gets adopted per domain if calibration data justifies it
- TR-8.5 — The evaluator (judge) version is versioned and stored alongside every verdict, so historical results remain interpretable even after the judge itself is recalibrated or upgraded

3.6 Cross-Platform Consistency Engine (TR-5) [Future Scope — FS-03, blocked on FS-02]

The platform's clearest differentiator versus anything on the market — confirmed via research as a genuine industry whitespace — but it structurally cannot be built before at least two platform/web adapters exist for the same solution.

- TR-5.1 — For any solution with >1 entry in `deployed_surfaces[]` (TR-1.1, already populated from Phase 1), one canonical test set is run through every relevant adapter for that solution
- TR-5.2 — Responses normalized to a canonical result shape, then compared for semantic equivalence (via TR-4.3) rather than literal string match
- TR-5.3 — Divergence beyond a defined tolerance is reported as a first-class defect type (“cross-platform inconsistency”), distinct from a single-surface functional failure

3.7 Reporting & Traceability Dashboard (TR-6) [Phase 1, extended in Phase 3]

- TR-6.1 [Phase 1] — `test_runs` store: `test_id`, `solution_id`, `version`, `result`, `evaluator_details`, `confidence`, `iso25010_tags[]`, `timestamp`
- TR-6.2 [Phase 1] — Traceability view: per-solution matrix of requirement ID × latest test result, auto-populated as a byproduct of test authoring/execution — never a manually maintained document
- TR-6.3 [Phase 1] — ISO/IEC 25010 coverage view: per solution, which of the nine quality characteristics have test coverage and current pass rate
- TR-6.4 [Phase 1] — Role-based access: solution owners see their own solution(s) by default; QA/leadership see org-wide status
- TR-6.5 [Phase 1, populated meaningfully from Phase 2 onward] — Golden-set regression store: a versioned, permanently-growing set of test cases seeded from real production failures; regression gating is per-metric/per-rule-slice, not one blended score
- TR-6.6 [Phase 3] — Exportable compliance report (PDF/CSV) per solution per version

4. Non-functional requirements

Category	Requirement	Phase
Performance	A CI-gating subset of tests for a single solution completes within [X minutes]; full regression + adversarial suite completes within [X hours], run nightly/pre-release	Phase 3

Category	Requirement	Phase
Scalability	Architecture supports the Phase 1 pilot solution count without redesign, with a documented path to [N] solutions post-pilot	Phase 1
Reliability	Adapter-level retries with backoff before a hard failure is recorded (TR-3.10)	Phase 2
Security	Target credentials in a secrets manager, never plaintext; adversarial corpus under exploit-database-equivalent access controls (TR-9.3)	Phase 2
Auditability	Test results immutable once recorded; evaluator/judge version stored alongside every verdict (TR-8.5)	Phase 1/3
Extensibility	New adapters/evaluators added as plugins without modifying the core orchestrator (TR-3.1/TR-4.1)	Phase 2
Cost control	Tiered model routing for LLM-driven evaluation; performance-test cost caps	Phase 3 / Future Scope (FS-05)

5. Interfaces / API contracts (illustrative) [Phase 1–2]

```

POST /solutions
{ name, type, owner, endpoint_ref, deployed_surfaces: [], brd_uri, tsd_uri }

POST /test-runs
{ solution_id, version, suite: "full" | "regression" | "smoke" | "cross_platform" }
```

Adapter plugin protocol:

```

connect(solution_target) -> Session
send_input(session, input) -> None
get_response(session) -> RawResult
teardown(session) -> None
```

Evaluator plugin protocol:

```

evaluate(test_case, raw_result) -> Verdict
Verdict = { pass_fail, confidence, coverage_note, rationale, iso25010_tags }
```

6. Proposed technology stack

Layer	Proposed technology	Phase	Rationale
Orchestration	Python (pytest-based core)	Phase 1	Suits short-lived, stateless, parallel execution; a durable-workflow engine (Temporal) deferred until a genuinely long-running human-in-the-loop campaign type exists
Web/UI automation	Playwright	Phase 2	Enterprise default; auto-waiting reduces flakiness

Layer	Proposed technology	Phase	Rationale
API testing	Schemathesis	Phase 2	Spec-driven fuzzing over hand-written cases
LLM services	Claude API	Phase 1–2	Requirement extraction, LLM-as-judge, conversation simulation
Tracing	OpenTelemetry / OpenInference	Phase 2	Vendor-neutral instrumentation, hedges against AI-eval vendor consolidation
Storage	Postgres	Phase 1	Registry, requirements, test cases, results, golden sets
Dashboard	Internal web app	Phase 1	Or an extension of an existing internal BI/reporting tool — open question
CI/CD	Existing pipeline (GitHub Actions/Jenkins)	Phase 3	Tiered required checks
Secrets	Existing org secrets manager	Phase 2	
Visual regression	Applitools-style perceptual diffing	Future Scope	
Accessibility	axe-core	Future Scope (FS-06)	De facto engine underlying most other a11y tools
Performance	k6 + streaming-LLM sub-adapter	Future Scope (FS-05)	k6 is the trending 2025-2026 default; LLM load testing is a distinct discipline
Platform/bot adapters	Bot Framework TestAdapter, custom Bolt harness, Botium-pattern	Future Scope (FS-02)	No mature off-the-shelf harness exists for Teams-specific surfaces or Slack
Enterprise integrations	Testcontainers, Pact, WireMock	Future Scope (FS-09)	Standard async/event-driven integration testing pattern
Security/red-team	Static corpus (Phase 2) → hybrid with PyRIT/Giskard-pattern adaptive generation	Phase 2 / Future Scope (FS-08)	Static-only corpora go stale as models patch known attacks
RAG evaluation	RAGAS-style layered metrics	Future Scope (FS-01)	De facto standard, retrieval/generation split enables failure localization

7. Meta-testing: validating MTA itself [Phase 1 onward]

- Golden set validation — pilot MTA against solution versions with known-good and known-broken behavior across each solution type as it's onboarded; confirm MTA correctly passes/fails before extending to the next type
- Judge calibration as a standing subsystem (TR-8.x, Phase 3) — not a one-off spot-check
- Extraction accuracy — track false positive/negative rate of requirements extraction against human-reviewed ground truth, ongoing from Phase 1

- Adapter/evaluator protocol conformance testing [Phase 2] — a conformance suite any adapter/evaluator must pass, ensuring MTA's own first-party adapters are held to the same public interface offered to future contributors

8. Security and compliance considerations

- No production client data used in test scenarios without explicit approval — synthetic or anonymized data only [Phase 1]
- Role-based access control (TR-6.4) [Phase 1]
- Adversarial/red-team test corpora stored under exploit-database-equivalent access controls (TR-9.3) [Phase 2]: separate repository, least-privilege grants, encryption at rest, audit-logged access, severity tiering
- Accessibility findings outside automated-scan coverage never auto-pass; mandatory human/AT review [Future Scope — FS-06]

9. Deployment and CI/CD integration [Phase 3]

- MTA runs a fast, tiered required-check gate at PR/merge time (smoke + fast unit/contract checks)
- Full regression/adversarial suite runs nightly or pre-release, not on every commit
- Failing the gate blocks promotion unless explicitly overridden with an audited sign-off from [role/team]
- A single required-check-compatible status is emitted per invocation tier, not per adapter

10. Assumptions, dependencies, and open questions

- Depends on pilot solution teams exposing a stable, testable staging/sandbox environment (Phase 0)
- Depends on continued availability/licensing of Playwright, Schemathesis, and (later) k6/axe-core/Pact
- Open question: which LLM(s) are approved for the judge/extraction/simulator role, and the cost budget per test run, per tier
- Open question: where the registry and dashboard are hosted
- Open question: initial gold-set size and re-calibration cadence per domain (TR-8.1/8.2) — set jointly with the QA guild during Phase 0
- Open question: whether a durable-workflow engine (Temporal) is needed before a Future Scope item (e.g., a multi-hour RAG evaluation campaign) makes the case concrete

11. Appendix

Example requirement object

```
{  
  "requirement_id": "TSD-4.2.1",
```

```

"solution_id": "solution-042",
"solution_type": "agent",
"source_doc": "TSD v1.3, section 4.2",
"clause_ref": "4.2.1",
"description": "Agent must decline to answer questions outside its documented scope",
"priority": "high",
"suggested_test_type": "functional, edge_case",
"iso25010_tags": ["functional_suitability", "safety"]
}

```

Example test case

```

{
  "test_id": "tc-0917",
  "requirement_id": "TSD-4.2.1",
  "solution_id": "solution-042",
  "test_type": "edge_case",
  "input": "A question clearly outside the agent's documented scope",
  "expected_behavior": "Agent declines and redirects to an in-scope topic",
  "evaluation_method": "llm_judge",
  "iso25010_tags": ["functional_suitability", "safety"]
}

```

Example evaluator verdict with confidence signal

```

{
  "test_id": "tc-0917",
  "pass_fail": "pass",
  "confidence": 0.91,
  "coverage_note": "Single-judge mode (Phase 2 default)",
  "rationale": "Response declines and redirects per rubric",
  "iso25010_tags": ["functional_suitability", "safety"]
}

```

Future Scope example — cross-platform consistency test case (specified now, built under FS-03)

```

{
  "test_id": "tc-xp-014",
  "requirement_id": "TSD-4.2.1",
  "solution_id": "solution-042",
  "test_type": "cross_platform_consistency",
  "surfaces_tested": ["web", "teams", "slack"],
  "expected_behavior": "Semantically equivalent decline-and-redirect across all surfaces",
  "evaluation_method": "llm_judge_semantic_equivalence",
  "iso25010_tags": ["compatibility", "functional_suitability"]
}

```